



Set de Práctica 3

Objetivos

- Comprender la noción de balance en un árbol.
- Comprender el funcionamiento de los árboles rojo-negro e implementar operaciones básicas.
- Comprender la relación entre los árboles rojo-negro y los árboles 2-4.

Árboles auto-balanceados (AVL y Rojo-Negro)

Como vimos en el set pasado, los árboles binarios de búsqueda tienen la facultad de realizar búsquedas, inserciones y eliminaciones proporcionales a la altura del árbol. Esto resulta eficiente cuando se encuentra balanceado, ya que en estos casos la altura $h \in \mathcal{O}(\log n)$. Debido a la utilidad de esta propiedad, existen variantes de los ABB que se auto-balancean, y en este set implementarás dos de ellas: AVL y Rojo-Negro.

La forma en que se trabajará este set difiere sustancialmente de los anteriores: esta vez les entregamos una implementación de ABB basada en el set anterior, y su misión será modificarla para convertir el ABB en un árbol AVL y en un árbol Rojo-Negro, y usarlos para resolver los eventos. La implementación que te entregamos se encuentra en `src/bst.c` y `src/bst.h`, siendo este último el que contiene la definición de los `structs` que componen al ABB:

```
bst.h
1 typedef struct bst_node {
2     int key;
3     struct bst_node *parent;
4     struct bst_node *left;
5     struct bst_node *right;
6 } BstNode;
7
8 typedef struct bst {
9     BstNode *root;
10    size_t size;
11 } Bst;
```

Como podemos ver, `BstNode` se compone de los elementos de un nodo descritos anteriormente, donde la `key` es de tipo `int`. Por otro lado, vemos que un `Bst` como tal se compone simplemente de una referencia al nodo raíz y una variable `size` que indica la cantidad de nodos dentro del árbol.

Por otro lado, dispondrás de varias funciones base para trabajar con esta implementación (los nombres son autodescriptivos): `bst_node_create`, `bst_node_destroy`, `bst_create`, `bst_destroy`, `bst_print`, `bst_search`, `bst_insert`, y `bst_delete`.

Dado que deberás implementar tanto la variante AVL como Rojo-Negro, puedes modificar como gustes la implementación base que te otorgamos; lo importante es que tu versión pase los tests correctamente. Además, en vista de esto, esta vez probablemente debas modificar el archivo `src/main.c` también para acomodar estos cambios y **manejar el output de los eventos**. Por último, cabe destacar que a veces en los archivos de input se llamará al evento LOG, esto gatillará una llamada a `bst_print` para ver que la estructura del árbol sea la esperada.

Ejercicios

BALANCE {X}

Se te entregarán a continuación X números enteros, con los cuales deberás armar un ABB, insertándolos en el orden en que se entregan. Luego, deberás balancear el árbol resultante, de modo que quede equilibrado en altura tal como lo estaría un árbol AVL. Es decir, el árbol resultante deberá mantener el mismo conjunto de *keys*, la propiedad de BST, y que para cada nodo, la diferencia de altura entre su subárbol izquierdo y su subárbol derecho sea a lo más 1.

Tras balancear el árbol deberás mostrar el siguiente output:

output
1 Se ha balanceado el ABB.

INSERT {X}

Deberás encargarte de insertar el número X en el árbol rojo-negro, manteniendo sus propiedades (colores y rotaciones según corresponda).

Tras insertar el número deberás mostrar el siguiente output:

output
1 Se ha insertado el numero {X} en el arbol rojo-negro.

DELETE {X}

Deberás encargarte de buscar y eliminar el número X en el árbol rojo-negro, manteniendo las propiedades de la estructura tras la eliminación.

1. En caso de eliminar exitosamente el número deberás mostrar el siguiente output:

output
1 Se ha eliminado el numero {X} en el arbol rojo-negro.

2. En caso de no poder eliminar el número (no se encontró) deberás mostrar el siguiente output:

output
1 No se pudo eliminar el numero {X} en el arbol rojo-negro.

EQUIV

Deberás mostrar el árbol 2-4 equivalente al árbol rojo-negro actual, donde cada nodo negro junto a sus hijos rojos directos forma un único nodo del árbol 2-4, compuesto por 1, 2 o 3 llaves.

Para el output deberás mostrar los grupos nivel a nivel, de izquierda a derecha, indicando en cada línea las llaves que componen el grupo en el orden en que aparecen en el árbol, separadas por un espacio:

output	
1	Arbol 2-4 equivalente al arbol rojo-negro:
2	{grupo_0}
3	{grupo_1}
4	.
5	.
6	.
7	{grupo_m}

Ejecución

Tu programa se debe compilar con el comando **make** (puedes limpiar el output de compilación con **make clean**) y debe generar un ejecutable de nombre **set-3** que se ejecuta con el siguiente comando:

```
./set-3 INPUT_PATH OUTPUT_PATH
```

donde **INPUT_PATH** será la ruta a un archivo con los eventos a simular y **OUTPUT_PATH** la ruta del archivo donde se guardarán los resultados. Por ejemplo:

```
./set-3 tests/input/test_01.txt outputs/test_01.txt
```

Si quieres verificar que el output generado por tu código es igual al esperado en un test (es decir, tu código pasa el test), puedes hacerlo con el siguiente comando:

```
diff PROGRAM_OUTPUT_PATH EXPECTED_OUTPUT_PATH
```

donde **EXPECTED_OUTPUT_PATH** será la ruta a un archivo con el output esperado del test, y **PROGRAM_OUTPUT_PATH** la ruta del archivo con el output de tu programa para dicho test. Si este comando finaliza sin imprimir nada en consola, significa que los archivos que entregaste son idénticos, y por tanto que el test pasa correctamente. En caso contrario, lo que se imprime es la diferencia entre los dos archivos, lo que podría ayudarte a ver porqué el test falló.

En caso de querer correr el programa para ver los leaks y errores de memoria utilizando **valgrind**, deberás utilizar el siguiente comando:

```
valgrind ./set-3 INPUT_PATH OUTPUT_PATH
```

Por último, en caso de querer correr todos los tests:

```
make test
```

A continuación detallamos los archivos de input y output.

Input

El archivo de input comenzará con el número **E** que corresponde al número de eventos a recibir, y las siguientes líneas corresponden a cada uno de estos eventos. Notar que el evento **BALANCE** ocupa más de una línea, ya que tras el comando se entregan los **X** elementos correspondientes. Como ejemplo tenemos el siguiente input:

input	
1	11 # Cantidad de eventos
2	BALANCE 5
3	4
4	2
5	6
6	1
7	3
8	INSERT 41
9	INSERT 38
10	INSERT 31
11	INSERT 12
12	INSERT 19
13	INSERT 8
14	EQUIV
15	DELETE 8
16	EQUIV
17	DELETE 100

Output

output

```
1 Se ha balanceado el ABB.
2 Se ha insertado el numero 41 en el arbol rojo-negro.
3 Se ha insertado el numero 38 en el arbol rojo-negro.
4 Se ha insertado el numero 31 en el arbol rojo-negro.
5 Se ha insertado el numero 12 en el arbol rojo-negro.
6 Se ha insertado el numero 19 en el arbol rojo-negro.
7 Se ha insertado el numero 8 en el arbol rojo-negro.
8 Arbol 2-4 equivalente al arbol rojo-negro:
9 19 38
10 8 12
11 31
12 41
13 Se ha eliminado el numero 8 en el arbol rojo-negro.
14 Arbol 2-4 equivalente al arbol rojo-negro:
15 19 38
16 12
17 31
18 41
19 No se pudo eliminar el numero 100 en el arbol rojo-negro.
```